

Souvenirs

Amaru koopt souvenirs in een vreemd winkeltje. Er zijn N **types** souvenirs. Van elk type souvenir zijn er oneindig veel beschikbaar in de winkel.

Elk type souvenir heeft een vaste prijs. Namelijk, een souvenir van type i ($0 \leq i < N$) heeft een prijs van $P[i]$ geldstukken, waar $P[i]$ een strikt positief geheel getal is.

Amaru weet dat de types souvenirs in aflopende volgorde van prijs geordend zijn, en dat de prijzen allemaal verschillend zijn. Specifiek geldt dat $P[0] > P[1] > \dots > P[N-1] > 0$. Hij kon ook de waarde van $P[0]$ te weten komen. Helaas heeft Amaru geen verdere informatie over de prijzen van de souvenirs.

Om wat souvenirs te kopen zal Amaru een aantal transacties met de verkoper begaan.

Elke transactie bestaat uit de volgende stappen:

1. Amaru geeft een (positief) aantal geldstukken aan de verkoper.
2. De verkoper plaatst deze stukken in een stapel op een tafel achterin, waar Amaru ze niet kan zien.
3. De verkoper overweegt elk type souvenir een voor een, in de volgorde $0, 1, \dots, N-1$. Elke type wordt **exact één keer** overwogen per transactie.
 - Bij het overwegen van souvenir type i , als het huidige aantal geldstukken op de stapel minstens $P[i]$ is, dan
 - verwijdert de verkoper $P[i]$ geldstukken van de stapel, en
 - plaatst de verkoper één souvenir van type i op de tafel.
4. De verkoper geeft Amaru alle geldstukken die op de stapel overblijven en alle souvenirs op de tafel.

Merk op dat er geen geldstukken of souvenirs op de tafel liggen voor het begin van een transactie.

Jouw taak is om Amaru instructies te geven voor een aantal transacties, zodat hij:

- in elke transactie **minstens één** souvenir koopt, en
- in totaal **exact** i souvenirs van type i koopt, voor elke i dat $0 \leq i < N$. Merk op dat dit betekent dat Amaru geen souvenirs van type 0 mag kopen.

Amaru hoeft het aantal transacties niet te minimaliseren en heeft een onbeperkte voorraad aan geldstukken.

Implementatie Details

Je moet de volgende procedure implementeren.

```
void buy_souvenirs(int N, long long P0)
```

- N : het aantal types souvenirs.
- $P0$: de waarde van $P[0]$.
- Deze procedure wordt voor elk testgeval precies één keer aangeroepen.

De bovenstaande procedure kan de volgende procedure aanroepen om Amaru opdracht te geven een transactie uit te voeren:

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- M het aantal geldstukken dat Amaru aan de verkoper heeft gegeven.
- De procedure geeft een paar terug. Het eerste element van het paar is een array L , met de soorten souvenirs die zijn gekocht (in oplopende volgorde). Het tweede element is een geheel getal R , het aantal munten dat Amaru na de transactie terugkreeg.
- Je moet zorgen dat $P[0] > M \geq P[N - 1]$. De voorwaarde $P[0] > M$ zorgt ervoor dat Amaru geen souvenir koopt van het type 0, en $M \geq P[N - 1]$ zorgt ervoor dat Amaru minstens één souvenir koopt. Als niet aan deze voorwaarden wordt voldaan, krijgt je oplossing de uitspraak `Output isn't correct: Invalid argument`. Merk op dat in tegenstelling tot $P[0]$, de waarde van $P[N - 1]$ niet in de invoer wordt gegeven.
- De procedure kan maximaal 5000 keer worden aangeroepen in elk testgeval.

Het gedrag van de grader is **niet adaptief**. Dit wil zeggen dat de reeks prijzen P bepaald zijn voor `buy_souvenirs` aangeroepen wordt.

Beperkingen

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$ voor alle i zodat $0 \leq i < N$.
- $P[i] > P[i + 1]$ voor alle i zodat $0 \leq i < N - 1$.

Subtaken

Subtaak	Score	Extra beperkingen
1	4	$N = 2$
2	3	$P[i] = N - i$ voor alle i zodat $0 \leq i < N$.
3	14	$P[i] \leq P[i + 1] + 2$ voor alle i zodat $0 \leq i < N - 1$.
4	18	$N = 3$
5	28	$P[i + 1] + P[i + 2] \leq P[i]$ voor alle i zodat $0 \leq i < N - 2$. $P[i] \leq 2 \cdot P[i + 1]$ voor alle i zodat $0 \leq i < N - 1$.
6	33	Geen extra beperkingen.

Voorbeeld

Beschouw de volgende aanroep.

```
buy_souvenirs(3, 4)
```

Er zijn $N = 3$ soorten souvenirs en $P[0] = 4$. Merk op dat er slechts drie mogelijke reeksen prijzen P zijn: $[4, 3, 2]$, $[4, 3, 1]$, en $[4, 2, 1]$.

Neem aan dat `buy_souvenirs` een aanroep `transaction(2)` maakt. Veronderstel dat de aanroep $([2], 1)$ teruggeeft, wat wil zeggen dat Amaru één souvenir van type 2 kocht en de verkoper hem 1 geldstuk teruggaf. Merk op dat dit toelaat te besluiten dat $P = [4, 3, 1]$ sinds:

- Voor $P = [4, 3, 2]$, zou `transaction(2)` de waarde $([2], 0)$ hebben teruggegeven.
- Voor $P = [4, 2, 1]$, zou `transaction(2)` de waarde $([1], 0)$ hebben teruggegeven.

Dan kan `buy_souvenirs` een aanroep `transaction(3)` maken, wat $([1], 0)$ teruggeeft, wat wil zeggen dat Amaru één souvenir van type 1 kocht en de verkoper hem 0 geldstukken teruggaf. In totaal heeft hij tot nu toe één souvenir van type 1 en één souvenir van type 2 gekocht.

Tot slot kan `buy_souvenirs` een aanroep naar `transaction(1)` maken, wat $([2], 0)$ teruggeeft, wat wil zeggen dat Amaru een souvenir van type 2 kocht. Merk op dat we hier ook `transaction(2)` konden gebruiken. Op dit punt heeft Amaru, in totaal, één souvenir van type 1 en twee souvenirs van type 2 gekocht, zoals gevraagd.

Voorbeeld Grader

Input format:

N

$P[0] \ P[1] \ \dots \ P[N-1]$

Output format:

$Q[0] \ Q[1] \ \dots \ Q[N-1]$

Hier is $Q[i]$ het aantal souvenirs van type i gekocht in totaal, voor alle i zodat $0 \leq i < N$.